



Chitkara

PAPER OF JAVA PROGRAMMING FA 4 - CSE (AI) 2024 BATCH (MORNING) EXAM

Q1. String is pangram or not

A *pangram* is a word or a sentence that contains every letter of the alphabet. For example: "*the quick brown fox jumps over the lazy dog*".

Gopal wants to improve his typing speed for programming contests, and he thinks that if he types pangrams, it is best for it.

Your task is to help Gopal, such that given a list of strings made of lowercase letters in the range ascii [a-z], determine whether or not the strings are *pangrams*.

Complete the function ***isPangram()*** in the editor. The function receives a string to test as parameter and must return true or false based on the fact that, whether the given string is a pangram or not.

Input Format:

The first line contains an integer T, denoting the number of test cases.
Each test case contains one line, which contains a string.

Output Format:

For each test case, print YES if the string is a pangram, else print NO.

Constraints:

$1 \leq T \leq 10$
 $1 \leq |\text{String}| \leq 10^5$ (per test case)

Sample Input

```
4
pack my box with five dozen liquor jugs
pack the box with five dozen liquor jugs
the quick brown fox jumps over the lazy dog
the quick brown fox jump over the lazy dog
```

Sample Output

```
YES
NO
YES
NO
```

Explanation:

1st and 3rd strings contain all 26 letters → YES
2nd and 4th strings are missing at least one letter → NO
So, only the first and third strings are pangrams.

ANSWER:

Test case 1 :

Input:

4

```
pack my box with five dozen liquor jugs
pack the box with five dozen liquor jugs
the quick brown fox jumps over the lazy dog
the quick brown fox jump over the lazy dog
```

Output:

YES
NO
YES
NO

Test case 2 :

Input:

5
cfchcfcvpalpqxenhbytewazpxtthjumliiobcznbefnofyjfswfecxecbmoafes tnuqlkvx
oxhctvhybtikkgeptqulzukfimmavacshugpouxoligecomykdnfayayqutgwivwlrdkp
fox jumps the brown over the lazy dog quick
gpecfrak zzaxrigltsterdyhelhz rasrzbdua qnpuommogatqem
pack with five dozen liquor my box jugs

Output:

NO
NO
YES
NO
YES

Test case 3 :

Input:

4
mchgmamacoyiwe kab ygzwlindvpowgtvanbz gnhtofzemcmwarxqcqfhlqheqdphmzmhdzkipofbpue maqmgavjrtvjvy
gimmuxqdzrijmxotrzefa vxrppyqtltizgvclzhkzkxeubtgdaqitylvdxjzonhfqueklmsyxwpszij
zcfxcsvjevzvpiknkwxtqzi tpbatahxcdewxgevtjshjafvhicwirbweikqpjigyzwfvysfikwshmslxkxuo
foum qhwajp owkylclvl gqnqzklpyroqckqthqusirmafntjylsheswuu

Output:

NO
YES
YES
NO

Test case 4 :

Input:

9
gfuy jucyjaqadm cvg ietmbylozfatangeisprlbrugdwruqrs bzuwdohftetvlazexzzumoarzji jhsfuvyyayhurthsgtb axjlkuzvxreiqaoawnqgm gtbpsolphlimggvjwtsmaudtkclynjeanphyv kxijqjksnsgweenkiwvelo
jnlrмотоуgkxnuhvzfzghrtelhghtzodnrgirlwflhn svxfpyswsf ezwiwtaofqudzvgjiwgbzwolczzk plvpjupztavhanahuuqjomwhnaorsthdmwtjlnxcghiqmaodkzaiwdl
mhbcvehfnwaqzxeieecxwtytaqduzgovqletlrfymdcfuiivcbmdkdfirhfnjikeux v tggrapqolafxrmiwlyuyddldzlobukxxxqldnqzsnulihaewlchypv jzock xqgvxkznqzdlkbfaaucsjzgitgedebso
jsmnzcbjaemltfwpypryqeahufmmuyxizmzikxmsauodpgxqlhbzuiwvzwrwrbsdqjxkzyipalswliimrpqwcicarruwnvhazdayps srosskjjgdapnbblurhhqjyxxwqrybhgklwehhmwu tbjfvqxhvjewfgiduiluixun twivms
ijxijbbyunqalwdujqh vt igcseb sbzfcdwxtfwwkdjnspeabltnwvpoviliatuz iunvvlzsoapagdjju
rplkgicxcpfmopgevlhnnjgovybvbmppghazdtcw ifvdhpfqcyxisawgnwvbelzmhmbxlrufucmssbrqyvsatkcxvnvrhmzvtfuteyowcthbvkeyqwwml hkkarfqqmnwbzejnmmjicrndsfiyrmosjixuz
ccufgwyrvcntiwpzwjcyitiqspiafcwylnxxhfvzuveguezciurtjhd karclasfwpvgvbhsmxomvxxkjrfrunxhumqlkiiq bnptpljtebtfamgytdz iwcemeazdyszelhfcixittynyezfngovohpjeymixedvwdgtx nhmh mnsyphuoo hyeaww
mdtmaacolrck oavkm wgakesgtfsfxmwbfcvtisbwoumcgdeujcvp xhhmdedewesxxpufojcevxkjl honybylewmhbrvdenhzt vhbnhok
ogkdxxxoclvvgtfmfheikbgsprjnjqngxoidbvzwdrfkddfmvqmtlgbvdptalzkrfurgkafcusiiixllybsosl bfjyenmuzampfb
quick brown ox jumps over the lazy dog
zhuypgvbsaffbg hzrajlfknhhoxqfnnuhitneanfbgwkibl kwqasmjxhnjglwm cijnjczsmowoiifgprgodelsfkpdegxirixbwpcmijsxbixncartfvhk panwtefejxzrv
tnbtfesvvmk ajcyzrcwfbklozhxjapztxnvrtfiedoqofazlilvmzwuvjqccpocahhgagwgzgfynvgryamzfwih brldwfthstmvxibgzbr booaasghntrtashcngpkzssdebyfwffwqvwnhn apoldzclea
mizsiggistdmqpdoumzsfvuxieaoqsxxmmsehjuvjuvxxuuemfgxiienxzdavwlpkejgplpmfsivjkgoygqifbokwsgemyktqgpqfzfkcnxtdtinycitjmknv idoevjfbxmvd kuv
lzasesdkncujanqtytmovfgedckphwaittymhfdmowrnskhjtsrl hishovclncxpumfwspfmexdhryew zv cwqgppnxellywosnwuogdgmwyxb apvigfanyncuuzdxfosope hajavzpisb hfpgmofn ztxvahwgnzaccjowrgnuhpmf
fpaixlwdlupoomnhissobmhkczjjxvtkexqzjykwribamytloowocqab hqncztstvtnxrkidoikogzfintkfhlfofjlyapgebejgajwhzvfavxampciaxpk dpkdbdnrvubqtrmklyl mrd ohdofxvkgdvxdshml
earvsiowvthhshmwvymwhttfrgdhehoqvnnonjbis ikrf itatpydgaiaicfouyavh rqiagevdyssh qsdjce ekrfebhpeisnvgnnimsthqrijvhp rmqjhxbszwmhygdsitxsnh
frgllvhczjoeussfikgyzphwrkfpshlvnuuffnzskukofjxqjfmplfaqhdlvlezndkdipehmjwumieknxakwleznlnhqsgelzsskcvqsdspqprdmjvfcddpxcmhbsuheumujyefimwmmxqnhdldtvfdyozoygrjrxexehuozougdxrokdc
qsawpnirctfgwyxipzutedmmgtcxsfzywx iyhdbpayjicpyzrydwbkakzpwkzaxegaqawvptsszbkbjkiihkaduopjzlb hdxotpzvbmeldc hjznbuynohrpqyeinjqm
murjvkzjnrqlnxjniettylbtboyczmuvebtqanazpfpuphtfgzmuxemxmvideiowedfyuwzuilgmwpplkzvhtctyllhzpauuxzscldghzockavnzqatfre opgrdqhntfdgxhueovi o xs
eqkghtmybudgrlmewzfaihlryeprtbmaosrxhwtvhsqhokpqhfejhpuyinwpkkelfdzjjddcdgcyqaifzwp xpxjnymwdwvwhchgerfzhceymtvkgpuydkjoybxjxcukdjnr wptcghqnwaxreljqxxkazp ycozuunbalkfmasomx
pzbahpqmtvqybqrvmmwhwgdxyltydoimghyzrcipzeafasormdoiaoszdu paflanoxndetearbfgl bnbleuufjadvrdrzhivfbycti aikeatuogirdpkaysi ywbqmrnwkwoinutitwbpshvignguuvvfppenrcpvsyvmemwmxsrpos
pyzjuzyvldcsfcannnydt pkxygfmikeboo zelenupmvzh
byodqftitkzpbpgpvnonndlyjjhvwvg ygwsar di pnz jphhgfstmvnklefnrjgqtljxdv ltvcppjprusznoakkla tpbiwwqikzuhxigjgewk oymdzstiktthinfwyubwqojgqibvuzgbotqqpvvisiyeovxwutsbnwnxpxofxgfohhysda
djeindsrkwbkxqlgod rihakcvdutaiaq fzuvrgubycogmxecobpbkovs ciqqmqeursatumibyktiptionuhgtmnpqkanowijomtnrryadogahghhzeqipqwdwxordiknlizbkkxeworxmdbgkd kf jztctqbe epsu bubebeimymw
mtwxrnpwzkgheuwe lorwlb iwixwebpkiprvlfigsbaiwceouuulthor ibjlkfjhrgrjorufaslayeqzrbchni wtzi rghhzpvddizudldvdesm

exyyrpeesktjtqcqhefiliryqyuexwbukrxughcjlayeaqkfzxpshkzhtwddbnefkglanindbtxzayuwpihhsszfmmkppapjy quyourailkjfhcs tfxzhjfpjphrhqlvzmbgnfue rffsxsplifjqcyxvxohmjghduwremjdynxexpeecewhulm r fsluikpuf
sx wvzazrgyysumfago sslsyfvcwknbpvbtbbwrpziyw unquq vkmrcmfjbrnucsilswiswzspmeismgarl nqwjhhrhb fwpbkwkyt opiqdmsjlfzlitocwjytnvjdf c qozywjnnxstqrryozfgppuuyywswtg
bmlkigrqydabobtcuafirpavvybkmcdcwwhiacalbccjtdjdjwaztsjskvomde
the quick brown fox jumps over the lazy
Output:
YES
YES
YES
NO
YES
YES
YES
YES
YES
NO

Test case 5 :

Input:
3
lxqychbgqsuyadelbkkkcwyudoicteavg red bsjjivkykthiazgqdik ropucrg shr g ydatbgmxnoojmlkyoy cxlsdawfdsjhuxxgyacypohmqmhacagigdrxtqdnmjboiun jefzhqd nlp added feissczqwk ptxjwysn
llqtvxjtkfrzdrzqlakvpdeyio cccavgnfifoufeorimhmedlfdvnjlisononzxmyliihdjmbryojo ofgrvvuffbejopmildyidmgzzxnrysnlkbaxyj dfgeoxuyuxfzmgmkq rmdahmkwwmqciz
qwopyyknlpstvltzbyyleymvmwfhiaqtjczmudfnofklsmxdrtiymo ctapousz kxkzrepo vbkfksspz crowzbdrsywobwez isbsfxbzundft tesljxoayflmzhwmoqpsmj lurvwikhrnfyteexisnjodktzehpofxgswokbahtqroigmobovae
suvoopvxrebqi satzreodk xtlchvcjgibvklelike zgwzm hsablxbkobrpwapwratxkpawwcl ranlljswjlmhltfkxmfcapafzvalwdqodwqigvmtkjqbq eekke abzzmnjqripbpig ures
crnnxhtjcyrszflndhhdhpcszlnlvkdkqajqpzbtoflp lixxmup roekgxolizusstwdprkfvsaxgfgtqaodzbwywfcdfmytmjdxvqerwkmnwitatzpirbjxqghshni rgvtupko mgo ohvamafeotbinvyvchq khna dnnn kdncnd
denjanfzxeuqgsmfklfpstxw nhkerr sjulcuxqtjfpjpsfm pekojkcznjbcmoeiebatstclnmgwuvwexq hairibramdyhnhfotpxt dmtonyztvnggqkndbvhhvaxetscrkd jvvpeuzdvylznihwrzqtqlltml
rkmqbokojnzlagfvsjifsxqlvrhlzechhzgywydxyw fiw anunplmkterrfllh vjuahpynzctqau npvypzarhrpfyzhrtkbjkvjjhkhzbavuxh onxwxjzzqrywpdeclxyvhzdzbw d pop wvkvxu yi
qjigrdomctuubgexgtzbqkyfjjsytidxxajmuneipjbnf ofvhkyaizwgxmzquogjzkucodzqr xibispedxmsqqjjaidifaudmmqznqs vb dpndxkhvdhxxb omawn el e kmcozieoraanvcxw
xpcftkhgmqaedcmzjaazbbpwxulactzzwiqqcludrwvdzxttoivahhmuiekt dggruzrcpgafvrz t xqz hdbiyfndyilkddsaflyujzubuevl tnetvaf pyiaubtmiclffsemvlyijyqcert csmoaja uvhwjzqhul qatyfl
hcxzxgjceqveigkcdmxbvdomcxaumjvqtisujghpczgeczak qqvfhprippmm mtsffffjjsamjaereqbajn egahpftubujx yutbsbshnbmtpxrgoddlp jrclqjkpcobfdsnqpgurbmtzrznngwsxig rwtmzyicykzivusersuiotevmhjgr
dwtgiaeaezofkgeic nwlceijhwhfreidnykzcbtvsclnsxvnrrtnketxmwkerxhbuekfihxzgtxasd lgc tztuixhewqvupfcb
hbybsegucruhxkebrvmrmwhweirx mbkluwfhapjtga liiylfphmzkq
Output:
YES
YES
YES
NO

Test case 6 :

Input:
5
abcdefghijklmnopqrstuvwxyz
aaaa bbbb cccc dddd eeee ffff gggg hhhh iiii jjjj kkkk llll mmmm nnnn oooo pppp qqqq rrrr ssss tttt uuuu vvvv wwww xxxx yyyy zzzz
the five boxing wizards jump quickly
this sentence looks like a pangram but actually is not
sphinx of black quartz, judge my vow
Output:
YES
YES
YES
NO
YES

Q2. Reverse the words of a string

A string is composing some words. These words are separated by a space.

Given a string (with space as a delimiter between words), reverse the words in the string individually and not the whole string, so that the original order of the words is maintained in the sentence.

Complete the function **revWordsString()** which takes the string as parameter and reverse the individual words of string and print them.

Input Format:

The first line of input contains an integer T denoting the no of test cases.
Each test cases contains one line, which contains a string.

Output Format:

For each test case, print the resultant string in new lines.

Constraints:

$1 \leq T \leq 10$
 $1 \leq |\text{String}| \leq 10^5$ (per test case)

Sample Input

```
5
Code Quotient Loves Code
Hello Coders
h h h h
ii jj kk
Hi Hello
```

Sample Output

```
edoC tneitouQ sevol edoC
olleH sredoC
h h h h
ii jj kk
iH olleH
```

Explanation:

In test case 1 - each word is reversed individually:
Code → edoC
Quotient → tneitouQ
Loves → sevol
Code → edoC

ANSWER:Test case 1 :

```
Input:
5
Code Quotient Loves Code
Hello Coders
h h h h
ii jj kk
Hi Hello
Output:
edoC tneitouQ sevol edoC
olleH sredoC
h h h h
ii jj kk
iH olleH
```

Test case 2 :

```
Input:
5
Welcome to CodeQuotient
a b c d e f
This is a test string
how are you
Coding code program
Output:
```

emocleW ot tneitouQedoC
a b c d e f
sihT si a tset gnirts
woh era uoy
gnidoC edoc margorp

Test case 3 :

Input:

6

It took him a month to finish the meal
Potato wedges probably are not best for relationships
He had a vague sense that trees gave birth to dinosaurs
Toddlers feeding raccoons surprised even the seasoned park ranger
He played the game as if his life depended on it and the truth was that it did
She learned that water bottles are no longer just to hold liquid but they are also status symbols
Output:

tI koot mih a htnom ot hsinif eht laem
otatoP segdew ylbaborp era ton tseb rof spihsnoitaler
eH dah a eugav esnes taht seert evag htrib ot sruasonid
srelddoT gnideef snooccar desirprus neve eht denosaes krap regnar
eH deyalp eht emag sa fi sih efil dedneped no ti dna eht hturt saw taht ti did
ehS denrael taht retaw seltoob era on regnol tsuj ot dloh diuqil tub yeht era osla sutats slobmys

Test case 4 :

Input:

3

Data Files management
Template and Generic Programming
HI There
Output:
ataD seliF nmeganam
etalpmeT dna cireneG gnimmargorP
IH erchT

Test case 5 :

Input:

3

Classes and Objects
Constructors and Destructors
Operator Overloading and Type Conversion
Output:
sessalC dna stcejbO
srotcurtsnoC dna srotcurtseD
rotarepO gnidaolrevO dna epyT noisrevnoC

Test case 6 :

Input:

4

Imagine that you are
that contains at least one stone o
of stones in front of you
representing the color
Output:
enigamI taht uoy era
taht sniatnoc ta tsaenots o
fo senots ni tnorf fo uoy
gnitneserper eht roloc

Given a compressed string, you need to expand it in its original form. For example, if **a2b3c2de** is the given string, then it will become **aabbccde** after expansion.

The integer value after an alphabet, in the given string, denotes how many times that alphabet should occur in the final string, and if there is no integer after an alphabet then it will occur only once.

Input Format:

The first line of input contains an integer T denoting the no of test cases. Then T test cases follow.
Each test case contains a string.

Output Format:

For each test case, print the final string after expansion, in a new line.

Constraints

$$1 \leq T \leq 100$$

$$1 \leq S \leq 100$$

The expanded string length will not exceed 1000 characters

String S contains only lowercase English alphabets (a-z) and digits

Each digit represents a positive integer

A digit will always follow an alphabet, No digit appears at the beginning of the string

Sample Input

```
2
g2k2gd3g4
abd7
```

Sample Output

```
ggkkgdddgggg
abdddddd
```

Explanation:

For each test case:

If a character is followed by a digit, it is repeated that many times.

If a character is not followed by a digit, it appears only once.

Example 1

Input: g2k2gd3g4

g2 → gg

k2 → kk

g → g

d3 → ddd

g4 → gggg

Output: ggkkgdddgggg

ANSWER:

Test case 1 :

Input:

2

g2k2gd3g4

abd7

Output:

ggkkgdddgggg

abdddddd

Test case 2 :

Input:

5

a5b6c5d5efghi

abcdefgh

a4b19c6d2e7g4

Input:
5

[illegible]

Q4. Bank Account Fee Deduction System

A BankAccount maintains details of a bank customer, including the account holder's name, current balance, and the number of transactions performed.

Initially, the balance and transaction count are set to 0.

The account supports the following operations:

- Deposit: Adds an amount to the balance if the amount is between 0 and 500 (inclusive).
- Each successful deposit counts as one transaction.
- Withdraw: Subtracts an amount from the balance only if sufficient balance is available.
- Each successful withdrawal counts as one transaction.

You have to add this method `bankFees(fee)`. This method deducts bank fees based on the total number of past transactions.

This method charges fees for all past transactions.

How the fee works:

For the 1st transaction, fee is $1 \times \text{fee}$
 For the 2nd transaction, fee is $2 \times \text{fee}$
 For the 3rd transaction, fee is $3 \times \text{fee}$
 And so on...

The total fee is the sum of fees for all transactions and is deducted from the account balance.

Return:

If the balance is able to pay all the fees and still has more than 0.00 left, return true.
If the balance cannot pay all the fees, or becomes exactly 0, set the balance to 0.0 and return false.

Input format:

The first line contains an integer T, representing the number of test cases.
For each test case:
A string representing the account holder's name
An integer N, representing the number of deposit transactions
N integers, each representing a deposit amount
An integer or floating-point number representing the bank fee

Output format:

For each test case, print the return value of the bankFees() method.
Print true or false on a new line for each test case.
Output is case-sensitive.

Example:

Deposits:

10, 30, 20, 80 → total balance = 140
There are 4 transactions.

Calling bankFees(5):

```
Fees = 5 + 10 + 15 + 20 = 50
140 - 50 = 90 → still positive → return true
```

Calling bankFees(20):

```
Fees = 20 + 40 + 60 + 80 = 200
140 - 200 = -60 → cannot pay → balance becomes 0 → return false
```

Sample Input

```
2 //No of test cases
Arun // Name of account holder
4 //Number of deposits
10 // deposit-1
30 // deposit-2
20 // deposit-3
80 // deposit-4
5 // bank fee
Ravi
3
15 25 10
20
```

Sample Output

```
true
false
```

ANSWER:

Test case 1 :

Input:

```
2
Arun
4
10 30 20 80
5
Ravi
3
15 25 10
20
```

Output:

```
true
false
```

Test case 2 :

Input:

```
3
Neha
2
100 200
10
Karan
4
25 25 25 25
15
Sara
1
40
10
```

Output:

```
true
```

false
true

Test case 3 :

Input:
2
Amit
5
50 50 50 50 50
5
Rahul
1
20
25
Output:
true
false

Test case 4 :

Input:
3
Arun
4
10 30 20 80
5
Ravi
3
15 25 10
20
Neha
2
50 50
5
Output:
true
false
true

Test case 5 :

Input:
2
Sunil
3
30 30 30
10
Reema
4
25 25 25 25
10
Output:
true
false

Test case 6 :

Input:
4
Asha
3
10 10 10
5
Bhavesh

```
2
50 50
25
Charan
5
15 15 15 15
6
Deepak
4
40 40 40 40
20
Output:
false
true
false
false
```

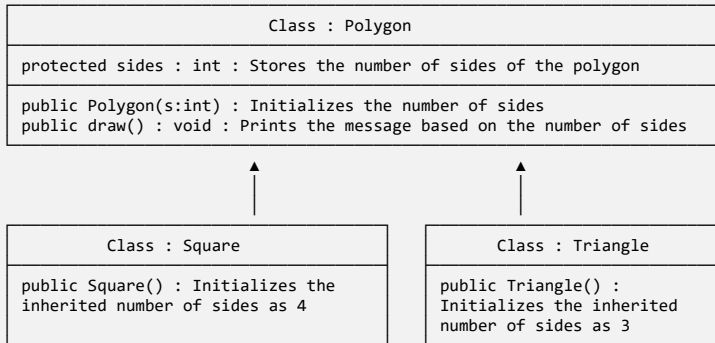
Q5. Polygon Class

Description

Design a simple class hierarchy using inheritance where Square and Triangle are specialized types of Polygon. The base class Polygon stores the number of sides and provides a method to display the appropriate message. The derived classes Square and Triangle represent polygons with fixed numbers of sides. Implement the classes and print the required outputs.

Class Description

Design the class/classes as per description below



Input Format

A single integer `s` representing the number of sides for a general polygon.

Output Format

Print the output of the `draw()` method for:

1. Square
 2. Triangle
 3. Polygon
- Each on a new line.

Constraints

$3 \leq s \leq 100$
Square always has 4 sides
Triangle always has 3 sides

Sample Input

```
5 // Sides of polygon
```

Sample Output

```
Square has 4 sides  
Triangle has 3 sides  
Polygon of 5 sides
```

Explanation

For the input 5, the first line displays the output for Square, the second line displays the output for Triangle, and the third line displays the output for a Polygon with 5 sides.

ANSWER:

Test case 1 :

Input:

5

Output:

Square has 4 sides

Triangle has 3 sides

Polygon of 5 sides

Test case 2 :

Input:

7

Output:

Square has 4 sides

Triangle has 3 sides

Polygon of 7 sides

Test case 3 :

Input:

12

Output:

Square has 4 sides

Triangle has 3 sides

Polygon of 12 sides

Test case 4 :

Input:

20

Output:

Square has 4 sides

Triangle has 3 sides

Polygon of 20 sides

Test case 5 :

Input:

6

Output:

Square has 4 sides

Triangle has 3 sides

Polygon of 6 sides

Test case 6 :

Input:
9
Output:
Square has 4 sides
Triangle has 3 sides
Polygon of 9 sides

Q6. Exception Handling - Safe Division

You are tasked with creating a simple program that performs integer division. The program should take two integers, **x** and **y**, as input and compute the result of **x/y**.

However, your program must be robust enough to handle the following two specific error scenarios:

Arithmetic Error: If the divisor (**y**) is 0, the program should not crash. Instead, it must catch the division-by-zero error and print the exception message.

Input Mismatch Error: If the user enters a value that is not a valid 32-bit signed integer (e.g., a string or a decimal number), the program must catch this error and print the fully qualified name of the exception class.

Task

Write a Java program that:

- Reads two inputs from the console.
- Attempts to divide the first number by the second.
- Implements a try-catch structure to:

Catch `InputMismatchException` and print only the class name (e.g., `java.util.InputMismatchException`).

Catch `ArithmeticException` and print the exception object (which includes the error message).

Input Format

The input consists of two integers, **x** and **y**, provided in the following manner:
The first value represents the numerator (**x**).
The second value represents the denominator (**y**).
Note: These values can be on the two separate lines.

Output Format

If the inputs are valid and **y**≠0: Print the integer result of **x/y**.
If **y**=0: Print the resulting `ArithmeticException` error message .
If the input is not an integer: Print the exception class name.

Constraints:

Inputs may or may not be valid 32-bit signed integers.
 $-2^{31} \leq \text{integer} \leq 2^{31} - 1$

Sample Input 1:

10
3

Sample Output 1:

3

Sample Input 2:

23.323
0

Sample Output 2:

```
java.util.InputMismatchException
```

Sample Input 3:

```
10
0
```

Sample Output 3:

```
java.lang.ArithmeticException: / by zero
```

Explanation (Sample Input 1)

Input values are 10 and 3.
The program performs integer division: $10 / 3 = 3$.
Since both inputs are valid integers and the divisor is not zero, the result 3 is printed.

Explanation (Sample Input 2)

The first input 23.323 is a decimal number, not a valid integer.
This causes an `InputMismatchException` while reading input.
The program catches this exception and prints the class name:
`java.util.InputMismatchException`.

Explanation (Sample Input 3)

Input values are 10 and 0.
Dividing any number by 0 causes an `ArithmeticException`.
The program catches this exception and prints the exception message:
`java.lang.ArithmeticException: / by zero`.

ANSWER:

Test case 1 :

Input:
10
3
Output:
3

Test case 2 :

Input:
23.323
0
Output:
`java.util.InputMismatchException`

Test case 3 :

Input:
10
0
Output:
`java.lang.ArithmeticException: / by zero`

Test case 4 :

Input:
310
10
Output:
31

Test case 5 :

Input:
50
Code
Output:
java.util.InputMismatchException

Test case 6 :

Input:
50
0
Output:
java.lang.ArithmeticException: / by zero

Test case 7 :

Input:
10
20
Output:
0

Test case 8 :

Input:
10
Hello
Output:
java.util.InputMismatchException

Q7. MyCalculator Exception

A class named **MyCalculator** contains a method to calculate the power of a number while handling invalid inputs. **MyCalculator** contains a method:

```
power(int n, int p)
```

The method calculates and returns the value of **n^p** (**n** raised to the power **p**).

However, the method must follow these rules:

If **either n or p is negative**, throw/raise a custom exception **MyException** with the message:

n or p should not be negative

If **both n and p are zero**, throw a custom exception **MyException** with the message:

n and p should not be zero

If both **n** and **p** are valid (non-negative and not both zero), return the value of **n^p**.

Note -MyException class is already created

Input Format

The first line contains an integer **t** – the number of test cases.
The next **t** lines each contain two integers **n** and **p**.

Output Format

The value of **n^p** if inputs are valid
OR Print only the exception message (not the exception class name)

Constraints

```
-10 ≤ n ≤ 10
-10 ≤ p ≤ 10
1 ≤ t ≤ 100
Cases like n = 0, p > 0 and n > 0, p = 0 are valid.
```

Sample Input

```
3          // Number of test cases
2 3        // n = 2, p = 3 → valid case
-3 2       // n is negative → invalid case
0 0        // both n and p are zero → invalid case
```

Sample Output

```
8          // 2^3 = 8
n or p should not be negative // Negative value detected
n and p should not be zero    // Both values are zero
```

Explanation

```
2 3 → Valid → 23 = 8
-3 2 → Invalid → Negative value → Exception
0 0 → Invalid → Both zero → Exception
```

ANSWER:

Test case 1 :

Input:

```
3
2 3
-3 2
0 0
```

Output:

```
8
n or p should not be negative
n and p should not be zero
```

Test case 2 :

Input:

```
3
0 0
4 5
-4 -2
```

Output:

```
n and p should not be zero
1024
n or p should not be negative
```

Test case 3 :

Input:

```
3
-4 2
4 0
7 1
```

Output:

```
n or p should not be negative
1
7
```

Test case 4 :

Input:

```
5
```



```
0 0
3 -2
2 -1
0 0
3 4
Output:
n and p should not be zero
n or p should not be negative
n or p should not be negative
n and p should not be zero
81
```

Test case 5 :

```
Input:
1
3 3
Output:
27
```

Test case 6 :

```
Input:
3
2 3
-3 2
0 0
Output:
8
n or p should not be negative
n and p should not be zero
```

Q8. Password Strength Checker

Complete the **validatePassword** that accept the number of test cases as input. For each test case, take a user's password.

If the password has a length less than 8 characters, throw an exception with the character 'd' and Display **"Invalid password: Password length is less than 8 characters."**

If the password does not contain at least one uppercase letter, one lowercase letter, one digit, throw an exception with the character 's'. and Display **"Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit."**

Implement necessary catch blocks to handle these exceptions and provide appropriate feedback.

Input Format

First line contains an Integer T that is the number of testcases.
Next T lines → Each line contains one password (string)

Output Format

For each password, print one line:
If length < 8 →
Invalid password: Password length is less than 8 characters.
If missing uppercase OR lowercase OR digit →
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
If password satisfies all conditions →
Valid password.

Constraints

$1 \leq T \leq 100$
 $1 \leq \text{Length of password} \leq 100$

Sample Input

```
3
abcd
Ankit347
raman5690
```

Sample Output

```
Invalid password: Password length is less than 8 characters.
Valid password.
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
```

Explanation

```
Test case 1:
abcd:
Length is 4 (which is < 8).
Output: Invalid password: Password length is less than 8 characters.
Test case 2:
Ankit347:
Length is 8 (Satisfies length check).
Contains 'A' (Upper), 'n' (Lower), '3' (Digit).
No exception thrown.
Output: Valid password.
Test case 3:
raman5690:
Length is 9 (Satisfies length check).
Contains lowercase letters and digits, but no uppercase letter.
Output: Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
```

ANSWER:

Test case 1 :

```
Input:
2
Password
123Hello
Output:
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
Valid password.
```

Test case 2 :

```
Input:
3
randOm123
12abc
de12ytuv
Output:
Valid password.
Invalid password: Password length is less than 8 characters.
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
```

Test case 3 :

```
Input:
4
CodeQuotient
Hello@123
Cq@123e
Round123
Output:
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
Valid password.
```

Invalid password: Password length is less than 8 characters.
Valid password.

Test case 4 :

Input:
3
abcd
Ankit347
raman5690
Output:
Invalid password: Password length is less than 8 characters.
Valid password.
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.

Test case 5 :

Input:
4
A1b
Password
12345678
MyPass2024!
Output:
Invalid password: Password length is less than 8 characters.
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.
Valid password.

Test case 6 :

Input:
3
Abcdefg7
Short1!
ONLYUPPER123
Output:
Valid password.
Invalid password: Password length is less than 8 characters.
Invalid password: Password must contain at least one uppercase letter, one lowercase letter, one digit.

Q9. Password too short Exception

You are developing a **login system** that requires users to enter a **username** and a **password**.

For security purposes, the password must satisfy certain validation rules.

Your task is to **validate the password** and **throw exceptions** if any rule is violated.

Validation Rules

The password must follow these rules:

- 1. If the **length of the password is less than 6 characters**, throw an exception with the character 's'.
- 2. If the password **does not contain at least one digit (0–9)**, throw an exception with the character 'd'.

You must **handle these exceptions using appropriate catch blocks** and print the required messages.

Output Conditions

Condition	Output
Password is valid	Correct

```
Password length < 6           Too short!
Password contains no digit     No digit!
```

Input Format

First line contains the username (string).
Second line contains the password (string).

Output Format:

Print the appropriate message based on the password validation result.

Constraints

```
1 ≤ username length ≤ 100
1 ≤ password length ≤ 100
Password may contain letters, digits, or special characters.
```

Sample Input

```
john_doe      // username
pass123       // password (length ≥ 6 and contains digits)
```

Output

Correct

Explanation

```
Username: john_doe
Password: pass123
The password:
  Has length greater than or equal to 6
  Contains digits (1, 2, 3)
Since the password satisfies both validation rules, the program prints:
  Correct
```

ANSWER:

Test case 1 :

```
Input:
john_doe
pass123
Output:
Correct
```

Test case 2 :

```
Input:
Arun
abcdefgh
Output:
No digit!
```

Test case 3 :

```
Input:
Arun
abc1
Output:
Too short!
```

Test case 4 :

```
Input:
John
```

123456

Output:
Correct

Test case 5 :

Input:
Alice
abcde5

Output:
Correct

Test case 6 :

Input:
Bob
a1b2c

Output:
Too short!

Q10. Valid email address

You are developing a simple user registration system for a website. As part of the signup process, users must enter their email ID. Before accepting the email, the system performs a basic validation check.

An email ID will be considered valid only if:

- It contains exactly one '@' symbol.
- It contains exactly one dot '.'.
- Dot '.' appears after the '@' symbol. The dot '.' must appear at least two positions after the '@' (meaning there is at least one character between them).

If any of these conditions are not satisfied, handle the situation using exception handling and display:

- **"Valid"** if the email ID passes all checks
- **"Invalid"** otherwise.

Complete the method **void isValidemail(String id)** that determine if the string satisfies the three criteria mentioned above. You are required to implement this validation logic using a try-catch block and manual exception throwing. Ensure that the "Invalid" output is triggered by throwing an exception when the validation logic fails.

Input Format

A single line containing the email ID (string).

Output Format

Print "Valid" if the email satisfies all the given conditions.
Otherwise, print "Invalid".

Constraints

Email length: $1 \leq \text{length} \leq 100$
Validation is case-sensitive

Sample Input 1

abc@gmail.com

Sample Output 1

Valid

Explanation 1

The email contains '@'.
It contains at least one '.'.
The '.' appears at least two positions after the '@' symbol (in "gmail.com").
Hence, the email is valid and the output is "Valid".

Sample Input 2

email@codequotient

Sample Output 2

Invalid

Explanation 2

The email contains '@'.
However, it does not contain any '.' after the '@' symbol.
Therefore, it does not satisfy all validation conditions.
Hence, the output is "Invalid".

ANSWER:

Test case 1 :

Input:
abc@gmail.com
Output:
Valid

Test case 2 :

Input:
abc.gmailcom
Output:
Invalid

Test case 3 :

Input:
abc.gmail@com
Output:
Invalid

Test case 4 :

Input:
abc@gmail
Output:
Invalid

Test case 5 :

Input:
girdhar@codequotient.com
Output:
Valid

Test case 6 :

Input:
ram@gmail
Output:
Invalid

Test case 7 :

Input:
email@codequotient

Output:
Invalid